
LABORATÓRIO DE PROGRAMAÇÃO

Sistema Especialista para Diagnóstico de **Falhas** **Automotivas**

Uma comparação entre paradigmas de programação.

Diele Ilana · Maria Eduarda · Matheus Macário · Rian Emmanoel



CONTEXTO

O problema e a **solução**

A identificação de falhas automotivas exige associar sintomas observados a possíveis problemas mecânicos, elétricos ou eletrônicos. Um diagnóstico rápido e confiável reduz custos e tempo de reparo.

Desenvolvemos um **sistema especialista** que recebe sintomas informados pelo usuário e retorna o diagnóstico mais provável e o componente a ser verificado.

MESMO PROBLEMA

Uma base de conhecimento com 10 regras de diagnóstico

3 PARADIGMAS

Orientação a Objetos (Python), Lógico (Prolog), Funcional (Lisp)

MESMOS TESTES

3 cenários idênticos para validar equivalência funcional

O PROBLEMA

De sintomas a diagnósticos

10

Regras na base

3

Paradigmas

3

Cenários de teste

Sintomas	Diagnóstico	Componente
Motor não liga + Luz bateria	Bateria descarregada	Bateria
Fumaça preta + Consumo alto	Mistura rica	Sist. injeção
Vibração excessiva + Perda potência	Problema na transmissão	Transmissão

Sintomas informados → Motor de Inferência → Diagnóstico + Componente

As Regras do Jogo: Três Cenários de Teste

Teste 1 (Correspondência Única)

motor_nao_liga + luz_bateria_acesa



Bateria Descarregada

Valida o caminho feliz.

Teste 2 (Correspondência Múltipla)

consumo_alto + luz_injecao_acesa + fumaca_preta



Falha na Injeção

Mistura Rica

Valida como o sistema lida com múltiplas respostas simultâneas.

Teste 3 (Falha Desconhecida)

pneu_furado

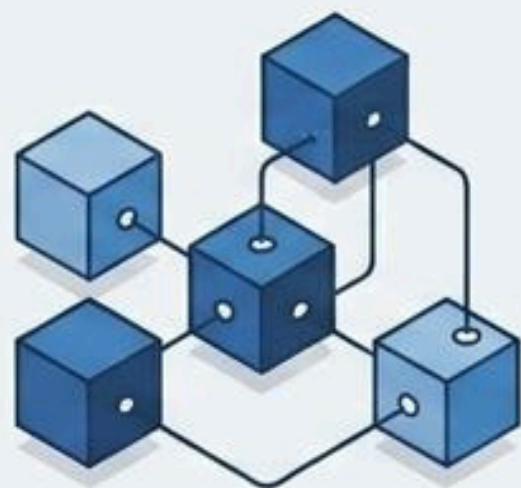


Valida o tratamento de exceção estrutural e cenários sem match.

Os Contendores: Não apenas sintaxe, mas Escolas de Pensamento

Python (Orientação a Objetos)

“Como fazer passo a passo.”



Foca em estado,
comportamento e
comportamento e modelagem
de entidades do mundo real.

Prolog (Programação Lógica)

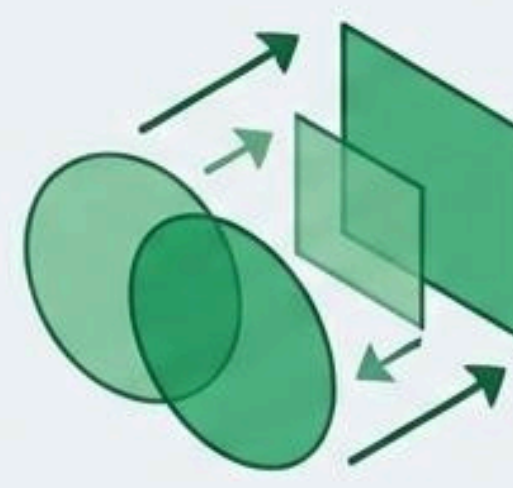
“Diga o que você quer, a máquina descobre como.”



Foca em fatos
declarativo, declarativos,
regras universais e
unificação matemática.

Common Lisp (Programação Funcional)

“Dados puros viram novos dados puros.”



Foca em imutabilidade,
ausência de efeitos
colaterais e
transformações recursivas.

Raio-X: Quem empurra as caixas?

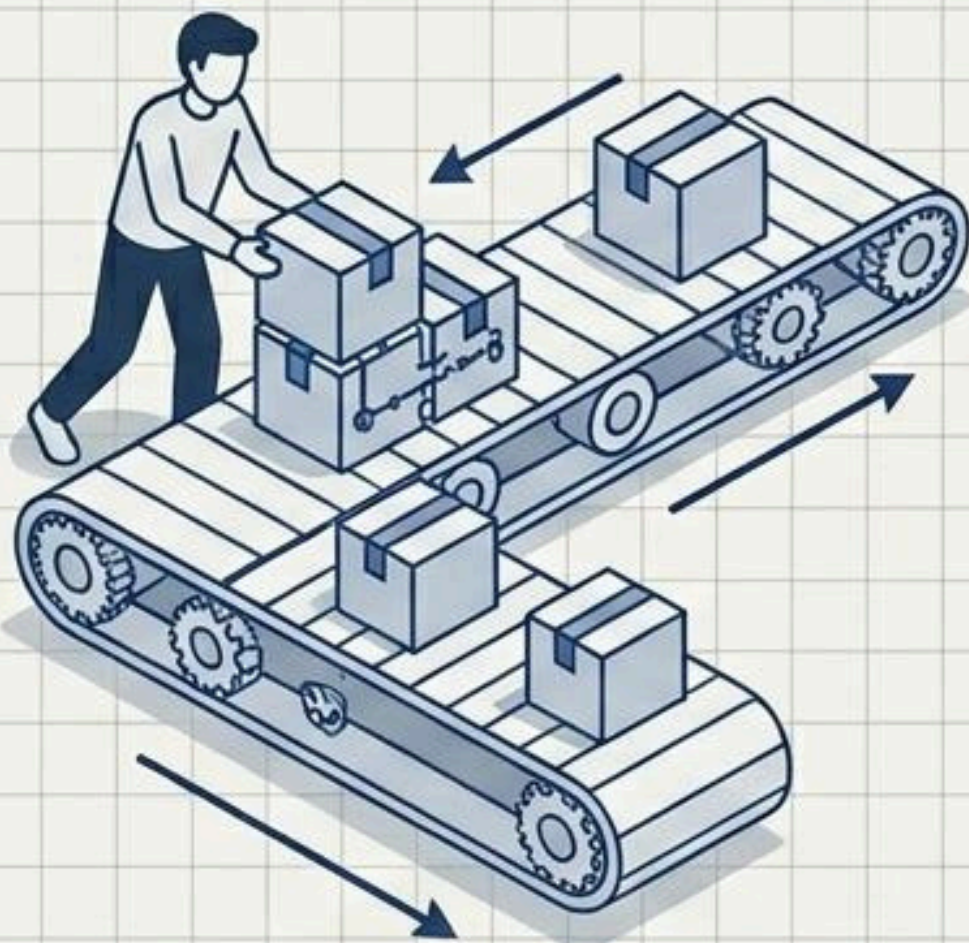


Nas **três linguagens**, o Teste 2 (múltiplos sintomas) exige que o código percorra toda a base de regras para não parar no primeiro match.

O resultado é idêntico nos três. Mas **COMO** cada linguagem encontra as respostas simultâneas revela o coração de sua filosofia.

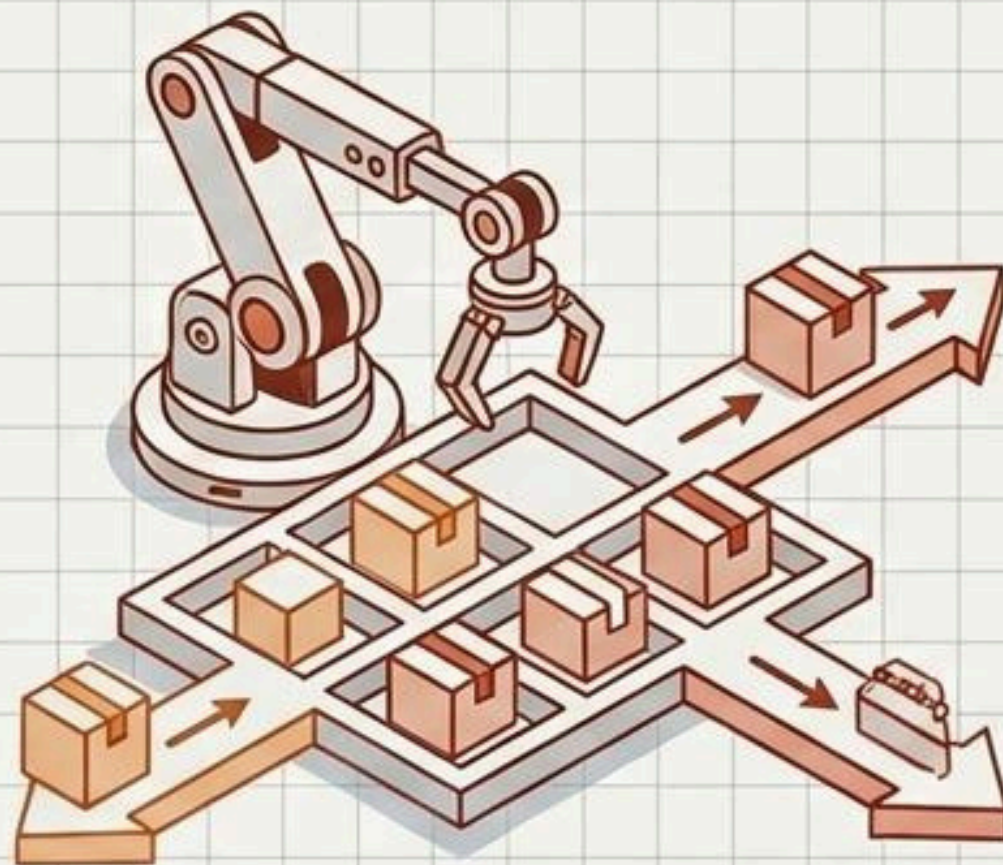
A Metáfora da Responsabilidade

Python



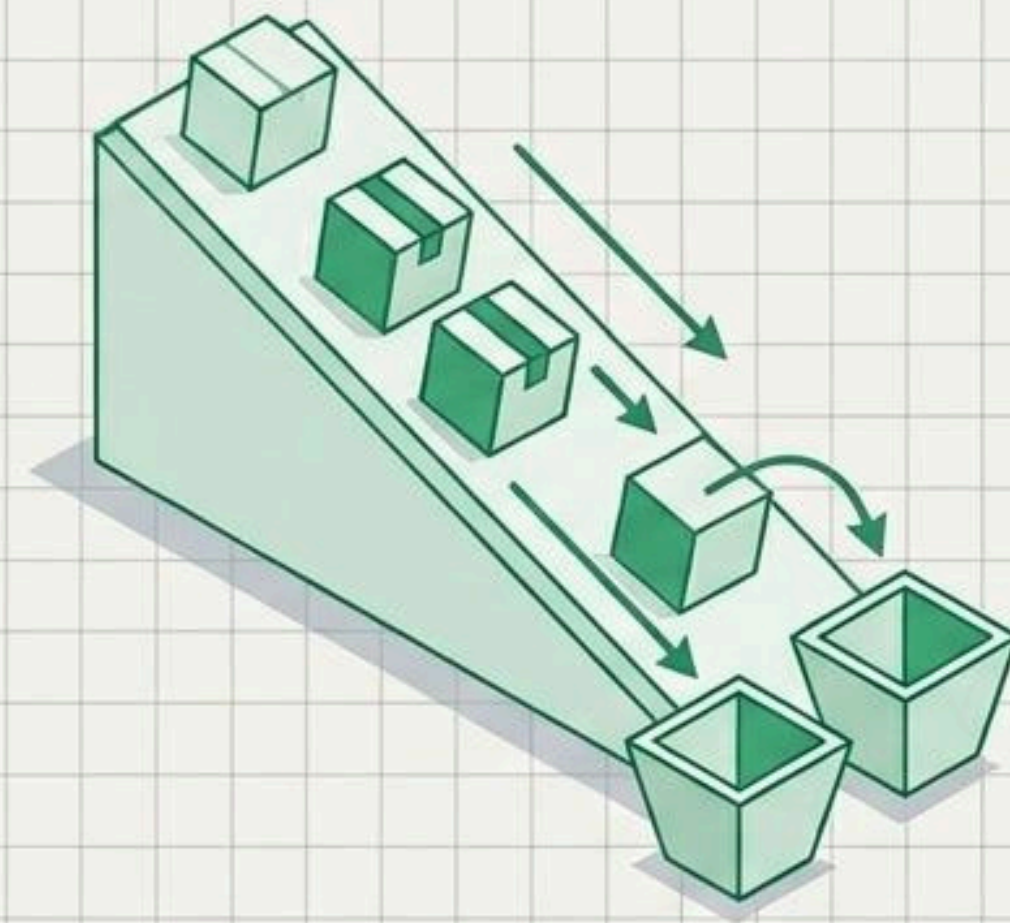
O desenvolvedor constrói manualmente a lógica de repetição e acumulação.

Prolog



O mecanismo nativo da linguagem assume todo o trabalho de busca profunda.

Lisp



Os dados fluem por transformações recursivas de forma fluida e sem atrito externo.

Python — Orientação a Objetos

O MOTOR DE INFERÊNCIA

```
def diagnosticar(self, sintomas_observados):  
    falhas_encontradas = []  
    for _, comp in self.componentes.items():  
        if comp.verificar_falha(sintomas_observados):  
            falhas_encontradas.append(  
                Falha(comp, comp.diagnostico))  
    return falhas_encontradas
```

O programador **constrói** o motor de inferência manualmente.

Prolog — Programação Lógica

REGRA + BUSCA

```
diagnostico(Sintomas, bateria_descarregada, 'Bateria') :-  
    member(motor_nao_liga, Sintomas),  
    member(luz_bateria_acesa, Sintomas).
```

```
consultar_falhas(Sintomas, Resultados) :-  
    findall([Diag, Peca],  
            diagnostico(Sintomas, Diag, Peca),  
            Resultados).
```

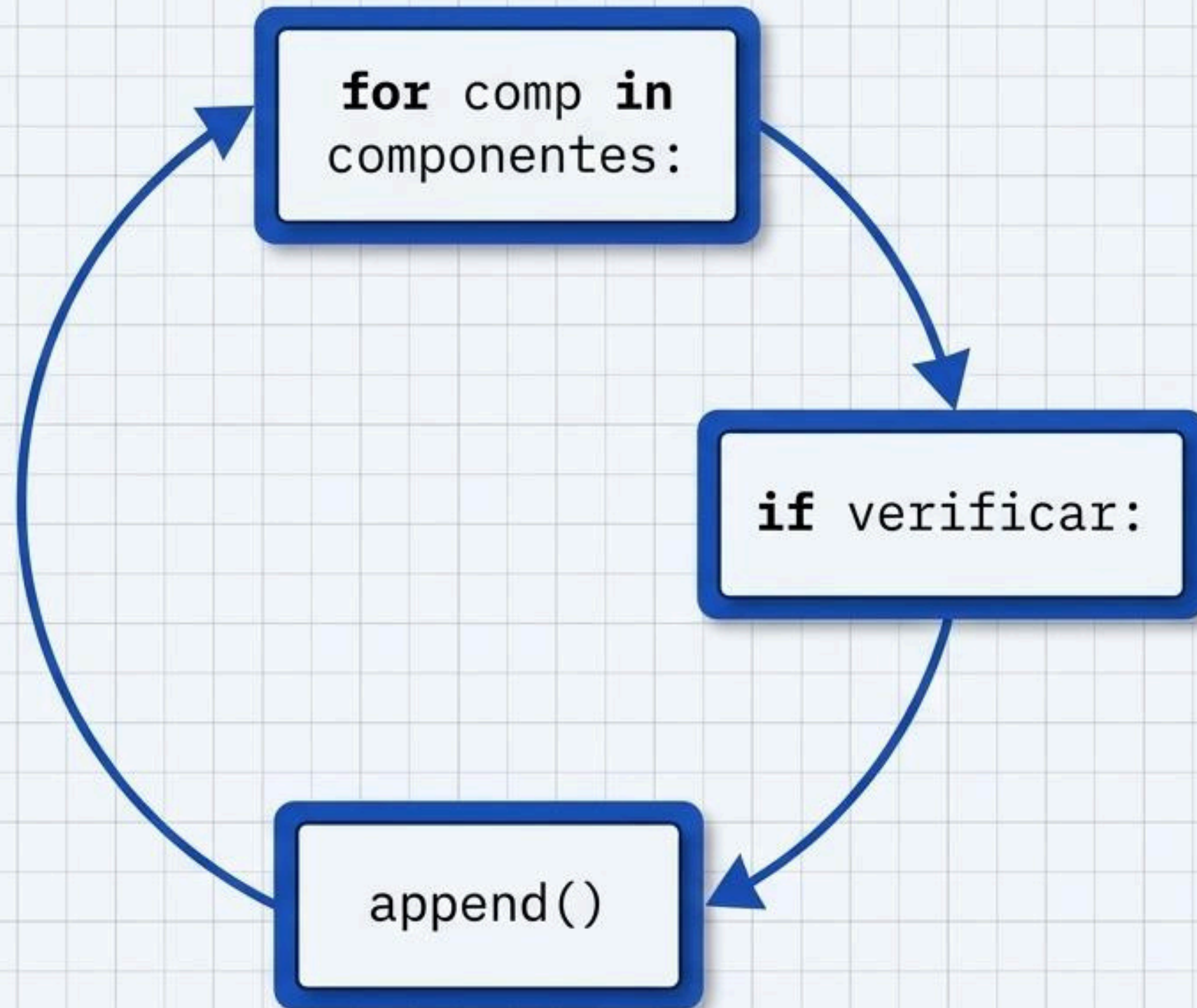
A linguagem **é** o motor de inferência.

Common Lisp — Programação Funcional

FILTRAGEM RECURSIVA

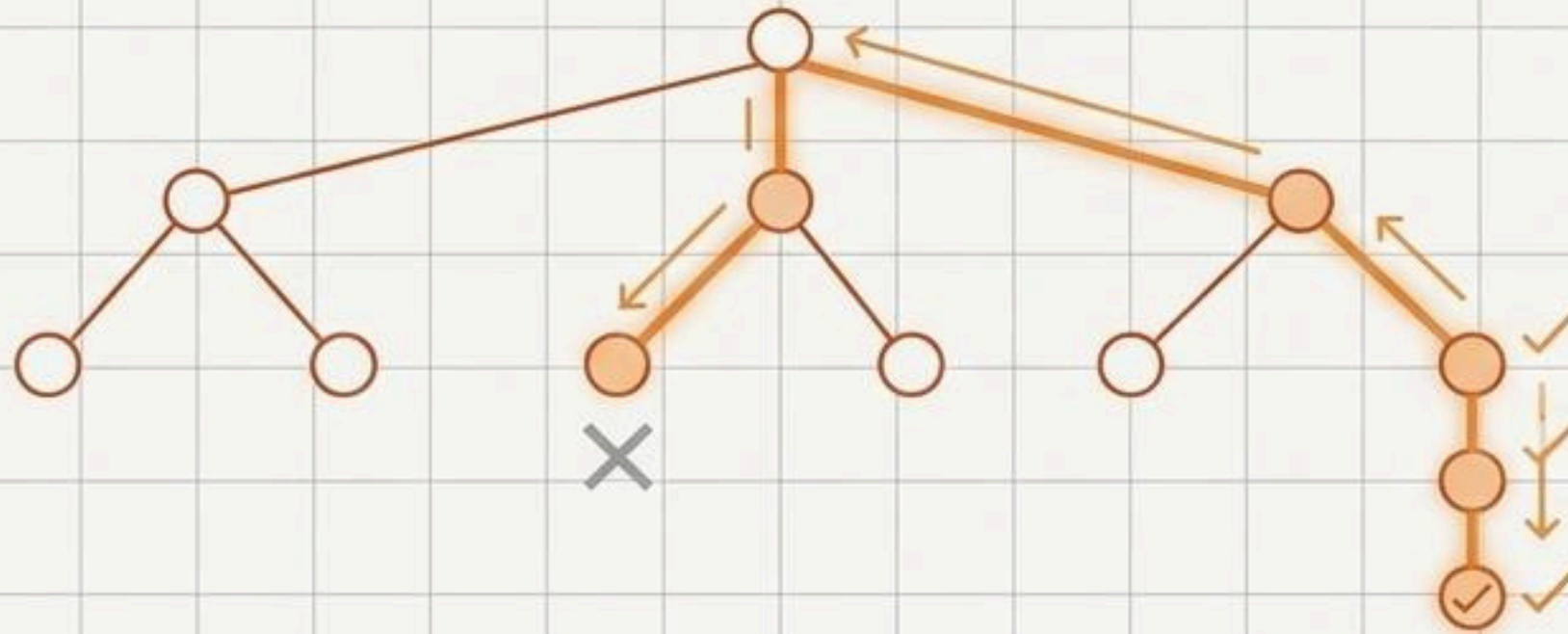
```
(defun filtrar-diagnosticos (sintomas regras)
  (cond
    ((null regras) nil)
    ((subconjunto-p (caar regras) sintomas)
     (cons (cdar regras)
           (filtrar-diagnosticos
            sintomas (cdr regras)))))
  (t (filtrar-diagnosticos
      sintomas (cdr regras)))))
```


O Mecanismo Imperativo (Loop Explícito)



A linguagem Python exige que o programador dite as ordens precisas: "Itere o dicionário", "Teste a falha", "Guarde em uma lista" e "Retorne o acumulador". A máquina não sabe o que está procurando, apenas executa os passos imperativos.

O Motor Declarativo (Backtracking Nativo)



```
findall(Diag, diagnostico(Sintomas, Diag, Peca), Resultados).
```

Em Prolog, o loop de iteração de componentes não existe no código. O predicado `findall` aciona o motor embutido da linguagem, que busca exhaustivamente todas as soluções possíveis pelo labirinto de regras.

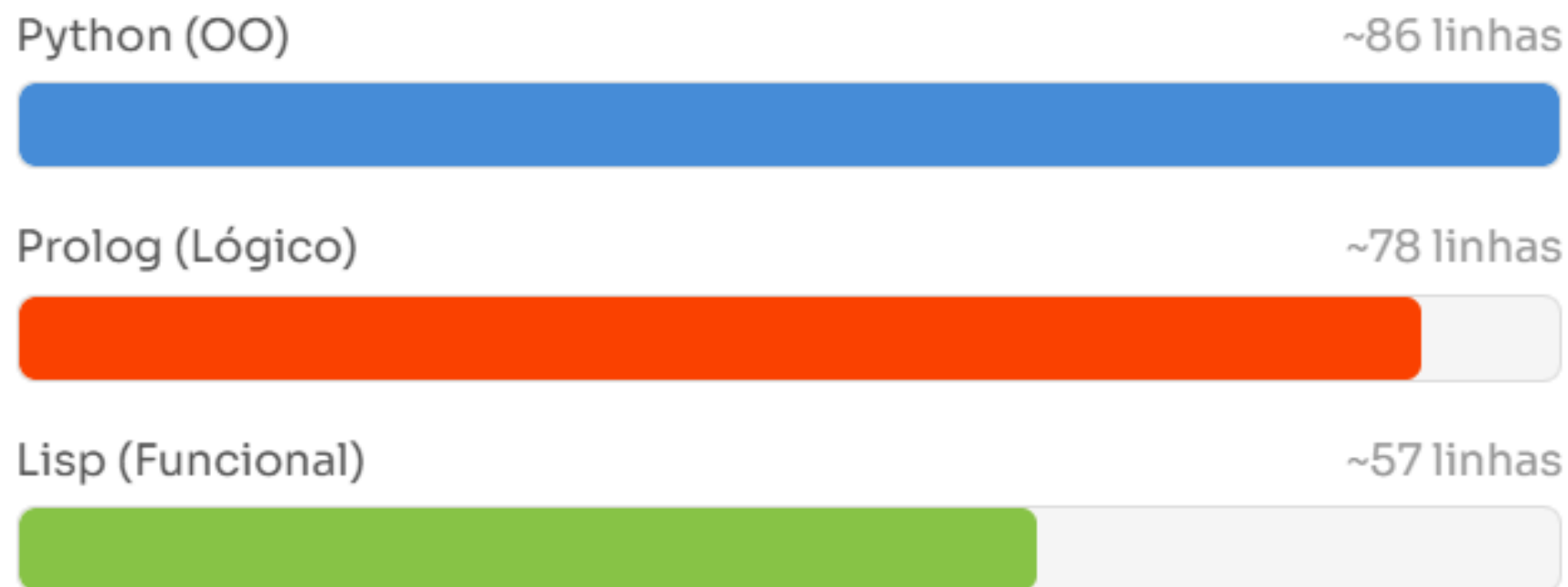
0 Fluxo Funcional (Recursão e Construção)



```
(cons  
  (cdar regras)  
  (filtrar-diagnosticos sintomas  
    (cdr regras)))
```

Não há comando de iteração e nenhuma lista acumuladora secundária é mutada. A função funde (**cons**) o match atual com o resultado da chamada recursiva do resto da base. **Imutabilidade absoluta.**

O Paradoxo do Volume de Código



Familiaridade
≠
Concisão

Apesar da equipe possuir maior domínio inicial sobre Python, a necessidade de escrever código ***boilerplate*** (classes, construtores, loops manuais) inflou o volume. O Lisp, com a curva de aprendizado mais alta e sintaxe menos familiar, resolveu o problema com a maior concisão, provando que expressividade supera familiaridade.

Tratamento de Corner Cases: A Filosofia do Vazio

Python

```
[]
```

Delega a decisão de mensagem de erro ("Falha desconhecida") para a camada superior de impressão. Típica separação de responsabilidades da OO.

Prolog

```
[[ 'Falha desconhecida',  
  'Nenhuma peca' ]]
```

Encapsula a exceção diretamente no domínio de saída lógico estruturado.

Lisp

```
(( 'Falha desconhecida...'  
  'Nenhuma peca...' ))
```

Consistência total do tipo de dado funcional de retorno em listas aninhadas.

A Matriz de Paradigmas

Aspecto	Python (OO)	Prolog (Lógico)	Lisp (Funcional)
Modelagem do Domínio	Classes e objetos	Fatos e regras	Listas e funções
Motor de Inferência	Manual (loop)	Nativo (backtracking)	Manual (recursão)
Expansibilidade	Alta (instanciar objeto)	Alta (adicionar regra)	Alta (adicionar à lista)
Volume de Código	Maior (~115 linhas)	Intermediário (~90 linhas)	Menor (~73 linhas)
Legibilidade	Alta	Média-Alta (declarativa)	Média (parênteses)
Adequação ao Problema	Boa	EXCELENTE	Boa

Cada paradigma no seu **lugar**

PYTHON — O MODELADOR

Organiza bem o domínio com classes e objetos, mas a hierarquia de herança aqui foi mais burocracia que necessidade. O motor de inferência é um loop manual.

PROLOG — O MAIS ADEQUADO

Foi feito pra esse tipo de problema. As regras viram predicados, o backtracking faz a busca sozinho. Zero lógica de iteração escrita pelo programador.

LISP — O MAIS CONCISO

A base de regras nunca é alterada. A função recebe dados, filtra por recursão e devolve dados novos. Menor código das três versões com 57 linhas.

Faria sentido uma única linguagem reunir os **três** **estilos**?

Se a Orientação a Objetos modela bem, o **Prolog** infere sem esforço, e o **Lisp** processa dados sem mutar nada... por que temos que escolher apenas um?

A Síntese Evolutiva: O Multiparadigma



TRADE-OFF

O Preço da Flexibilidade **Universal**

+

Ganhos

Flexibilidade extrema. Permite modelagem (OO) e processamento (Funcional) no mesmo arquivo, eliminando a 'impedância' de arquitetura entre camadas distintas do sistema.

—

Custos

Maior curva de aprendizagem. Existe um risco inerente de misturar estilos inconsistentemente, criando o temido "código macarrônico arquitetural".

MUNDO REAL

**Por que empresas escolhem
linguagens multiparadigma?**



CASO 1



100M+ clientes · Maior banco digital da América Latina

POR QUE FUNCIONAL?

Transações financeiras **não podem ter efeitos colaterais imprevisíveis**. Se uma operação de débito alterar acidentalmente o estado de outra conta, o banco quebra. A imutabilidade por padrão do Clojure elimina bugs de estado compartilhado — cada operação cria dados novos sem alterar os anteriores.

CASO 3



2 bilhões de usuários · ~50 engenheiros

POR QUE FUNCIONAL PURO?

Cada conversa roda em um **processo isolado e imutável**. Se um processo falha, os outros continuam sem saber — não há estado compartilhado para corromper. **Zero efeitos colaterais = zero corrupção de estado** entre bilhões de mensagens simultâneas.

CASO 2

Twitter / X Scala

Migrou de Ruby em 2009 · Escala para 500M+ tweets/dia

POR QUE OO + FUNCIONAL?

OO modela entidades (tweets, usuários, timelines) — cada uma com estado e comportamento.

Funcional transforma dados massivos com `map` / `filter` sem mutação. Pattern matching expressa regras de forma quase declarativa.

SÍNTESE

Paradigma é **decisão de arquitetura**

Cada empresa escolheu a linguagem que resolve o **problema central** do seu domínio:

FINANÇAS

Imutabilidade

Nubank → Clojure

ESCALA WEB

OO + Funcional

Twitter → Scala

MENSAGERIA

Concorrência

WhatsApp → Erlang

A escolha **não é preferência pessoal** — é a linguagem que **resolve o problema** do domínio.

CONCLUSÃO

A pergunta certa não é qual paradigma é melhor

É: **qual problema você está resolvendo?** O paradigma é consequência do domínio, não preferência do programador.

Inferência

→ Lógico (Prolog)

Modelagem

→ OO (Python)

Transformação

→ Funcional (Lisp)

Diele Ilana · Maria Eduarda · Matheus Macário · Rian Emmanoel

Laboratório de Programação — BICT/UFMA